

breadbuns

A command-line tool that locks and unlocks PDF files with Adobe's own native password protection, no Adobe software required to create the lock.

2026-09-17. Covers the current build of breadbuns in `/Users/wessch/Projects/Stack/breadbuns`. Written for someone running the compiled **breadbuns** binary from a terminal on macOS; no prior command-line experience assumed beyond opening the Terminal app. Does not cover the Go source code itself or how the encryption is implemented internally.

What this is. breadbuns looks at a folder you choose, finds the PDF files that need locking or unlocking, and does the whole batch after one password prompt. When it locks a file, the result is a genuinely password-protected PDF: opening it in Adobe Acrobat, Preview, or any other PDF reader shows that reader's own "enter password" screen, exactly as if you had protected the file from inside Acrobat itself. breadbuns just does not require Acrobat to be installed to make that happen.

What breadbuns does, in one pass

Two jobs, one tool. breadbuns only ever does one of two things in a run: it either **encrypts** a set of PDFs (adds a password) or **decrypts** a set of PDFs (removes a password). You choose which, each time you run it. It never mixes the two in a single pass, and it never touches a file you have not been shown and asked to confirm.

Encrypting makes a copy. When you lock `report.pdf`, breadbuns leaves the original exactly as it was and creates a new file, `report_locked.pdf`, which is the password-protected one. If `report_locked.pdf` already exists from an earlier run, a fresh encryption overwrites it. The unlocked original is never deleted, so you always have an unlocked copy sitting next to the locked one unless you remove it yourself.

Decrypting works in place. There is no copy involved in reverse: breadbuns removes the password from the file itself, in place, once you give the correct password. There is no "original" kept aside for a decrypt, since the encrypted file is the file being worked on.

WHERE IT FITS Use breadbuns when you need to send a batch of PDFs somewhere and want them to require a password to open, or when you have received locked PDFs of your own making and want the password removed. It is not a general PDF editor, viewer, or a way to add watermarks, permissions, or redactions.

Starting the program

Open a terminal. On a Mac, open the Terminal app (or iTerm, if that is what you use). breadbuns is a text-based program that reads your keyboard directly, so it needs to run in a real terminal window, not through a script that feeds it text from a file.

Run it. Type the following two lines and press return after each:

```
cd /Users/wessch/Projects/Stack/breadbuns
./breadbuns
```

If you have changed the code. Should the program ever need rebuilding after an edit to its source, run `go build -o breadbuns .` in that same folder first, then start it with `./breadbuns` as above. Ordinary use of the tool never requires this step.

Step 1 Choosing a folder

The folder browser. Instead of typing a path, breadbuns opens a full-screen list of the folder you started in, with its subfolders listed below it. This replaces having to remember or type `cd` commands.

CONTROLS Up and down arrow keys (or `j/k`) move the highlighted line. Enter opens a highlighted subfolder, or, if `[ Use this folder ]` is highlighted, selects the folder you are currently viewing. Backspace moves up to the parent folder. The `~` key jumps straight to your home folder. `q` or `Esc` cancels and quits the program entirely.

Confirming the choice. Once you press enter on `[ Use this folder ]`, breadbuns prints the folder path back to you and asks `"Use this folder? [Y/n]"`. Pressing return alone accepts it, since capital `Y` is the default. Typing `n` sends you back into the browser to pick again.

Step 2 Encrypt or decrypt

The menu. Next, breadbuns asks what you want to do, with a short numbered menu: 1 to encrypt, 2 to decrypt, or `q` to quit without doing anything. Type the number and press return.

Step 3 Reviewing the file count

What gets counted. breadbuns scans every `.pdf` file directly inside the folder you chose (it does not look inside subfolders) and works out which ones are relevant to the job you picked. For encrypting, that means unlocked PDFs that are not already named with a `_locked` suffix. For decrypting, it means any PDF that is actually password-protected right now, regardless of its filename.

The list you see. It then prints how many files matched and lists every one of them by name, so you can check the right files are about to be touched before anything happens. If any PDF in the folder could not be read at all (for instance, a corrupted file), it is listed separately as skipped rather than silently ignored.

NOTHING TO DO If no files match, breadbuns says so and stops there. Nothing is written to disk in that case.

Step 4 Confirming and entering a password

Last chance to back out. breadbuns asks `"Proceed with encrypting/decrypting these N file(s)? [Y/n]"` before it touches anything. Answering `n` cancels the whole run with no changes made.

One password, whole batch. After confirming, you are asked for a password once, and it is typed without echoing to the screen, the same way a login prompt hides your typing. That single password is used for every file in the batch: every file you are encrypting in this run gets locked with it, and every file you are decrypting in this run must already be protected with it.

ENCRYPTING ONLY When locking files, you are asked to type the password a second time to confirm it. If the two entries do not match, breadbuns stops and asks you to run it again rather than risk locking files with a password you did not mean to set.

Step 5 Reading the results

The report. Once processing finishes, breadbuns prints a one-line summary: how many files succeeded and how many failed. Below that, it lists the succeeded files by name (for an encryption run, this is the new `_locked.pdf` name), and, separately, any failures with a short reason next to each one, such as an incorrect password for a specific file during a decrypt run.

Warnings. A decrypt run may end with a third list headed *Warnings*. A file listed there *was* unlocked and is fine to use, but its lock settings did not pass an integrity check: the sealed record of the file's permissions, written when it was locked, no longer matches what the file claims. That usually means the file was edited by hand or by a tool that does not follow the PDF standard closely, not that the password was wrong. Treat it as information, not as a failure.

Partial success is normal. A batch is not all-or-nothing: if nine files lock correctly and one fails because it turned out to already be encrypted by something else, breadbuns still completes the other nine and reports the tenth as a failure rather than stopping the whole run.

Limitations worth knowing before you rely on this

PDF files only. breadbuns only understands PDF. Other Adobe file types such as Photoshop or InDesign documents are not supported and will not appear in any file count.

It unlocks modern locks only. Decrypting works on any PDF protected with AES-256, which is what breadbuns itself writes and what Adobe Acrobat produces when the compatibility setting is "Acrobat X and later". Files locked with the older RC4 or AES-128 methods (Acrobat's "Acrobat 7" and earlier settings, and many older tools) are recognised but not supported: they are reported as a failure with an "unsupported encryption" reason rather than opened.

There is no password recovery. The password is not stored anywhere by breadbuns, in the file or otherwise. If you lock a file and forget the password, there is no way to recover it, from breadbuns or from Adobe. Treat the password the same way you would treat any password you cannot afford to lose.

Top-level folder only. Only PDFs sitting directly inside the folder you select are processed; PDFs inside subfolders of that folder are left alone and not counted either way.

Needs a real terminal window. Because the folder browser reads arrow-key presses directly, breadbuns cannot be driven by piping text into it or by calling it from a script that is not itself running in an interactive terminal.

If something looks wrong. If a file you expected to see in the count is missing, check whether it is already named `_locked`, already password-protected, or sitting in a subfolder rather than the chosen folder directly, since all three are handled by design rather than by accident.